EgoTECH

SOFTWARE REQUIREMENTS SPECIFICATION

EgoTECH PDF Vault — Document & Image Storage System

Document Type	Software Requirements Specification (SRS)
Project Name	EgoTECH PDF Vault
Organization	EgoTECH
Version	1.0
Date	October 20, 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the EgoTECH PDF Vault system (version 1.0). It is intended for developers, testers, and project stakeholders.

1.2 Scope

EgoTECH PDF Vault is a web-based platform enabling authenticated users to upload, organize, search, download, and delete PDF documents and image assets. The system provides folder-based organization for PDFs and category-based organization for images, all protected by JWT authentication.

1.3 Definitions & Acronyms

Term / Acronym	Definition
API	Application Programming Interface
JWT	JSON Web Token — used for stateless authentication
SPA	Single-Page Application
ORM	Object-Relational Mapper (SQLAlchemy)
CORS	Cross-Origin Resource Sharing
ASGI	Asynchronous Server Gateway Interface
UUID	Universally Unique Identifier (used as primary keys)
PDF	Portable Document Format
Free Plan	User tier limited to 3 PDF uploads per day
Pro Plan	Upgraded user tier with unlimited uploads

1.4 References

- FastAPI Documentation — https://fastapi.tiangolo.com
- React Documentation — https://react.dev
- SQLAlchemy Documentation — https://docs.sqlalchemy.org
- Pydantic Documentation — https://docs.pydantic.dev

2. Overall Description

2.1 Product Perspective

EgoTECH PDF Vault is a standalone, self-hosted web application. It consists of two independent processes — a React frontend (served by Vite on port 5173) and a FastAPI backend (served by Uvicorn on port 8000) — communicating via a RESTful HTTP API.

2.2 Product Functions (Summary)

- User registration and login with JWT-based session management.
- PDF upload with magic-byte validation and size enforcement (max 25 MB).

- Folder management (create, list, delete) per authenticated user.
- Document management (list with search, download, delete) per folder.
- Image category management (create, list, delete).
- Image asset management (upload, list, serve, delete) per category.
- Tiered usage plan enforcement (Free: 3 uploads/day; Pro: unlimited).

2.3 User Classes

User Class	Description
Unauthenticated Visitor	Can only access the login/register page.
Free User	Authenticated; limited to 3 PDF uploads per day.
Pro User	Authenticated; unlimited PDF uploads.
Administrator	Direct database access; no dedicated admin UI in v1.0.

2.4 Operating Environment

- Server OS: Windows 10/11 or any Linux distribution with Python 3.10+.
- Client: Any modern web browser (Chrome, Firefox, Edge, Safari).
- Network: Localhost or LAN deployment; no internet required post-setup.

2.5 Constraints

- Maximum file upload size: 25 MB (configurable via MAX_UPLOAD_MB).
- PDF files must have a valid %PDF- magic-byte header.
- Images must be of type JPEG, PNG, GIF, WEBP, or BMP.
- SQLite is used as the database in v1.0; not suitable for high-concurrency production use.
- Plan quota (Free tier) is enforced via localStorage on the client; server-side enforcement is planned for v2.0.

3. Functional Requirements

3.1 Authentication Module

ID	Requirement	Priority
FR-AUTH-01	The system SHALL allow new users to register with a unique username, unique email, and a password of 8–128 characters.	High
FR-AUTH-02	The system SHALL hash all passwords using bcrypt before storage.	High
FR-AUTH-03	The system SHALL authenticate users by email and password and return a JWT bearer token on success.	High
FR-AUTH-04	The system SHALL reject login attempts with invalid credentials	High

	with HTTP 401.	
FR-AUTH-05	The system SHALL expose a /auth/me endpoint returning the authenticated user's profile.	Medium
FR-AUTH-06	The system SHALL reject duplicate username or email registrations with HTTP 409.	High
FR-AUTH-07	JWT tokens SHALL expire after a configurable period (default: 60 minutes).	High
FR-AUTH-08	The frontend SHALL detect token expiry and dispatch an auth-expired event, logging the user out.	High

3.2 Document (PDF) Module

ID	Requirement	Priority
FR-DOC-01	The system SHALL allow authenticated users to upload PDF files up to 25 MB.	High
FR-DOC-02	The system SHALL validate that uploaded files end with .pdf and begin with the %PDF- magic bytes.	High
FR-DOC-03	Uploaded PDFs SHALL be stored on the filesystem with a UUID-based filename to prevent conflicts.	High
FR-DOC-04	Each document record SHALL store: UUID, original filename, stored filename, size in bytes, upload timestamp, user_id, and folder_id.	High
FR-DOC-05	The system SHALL list all documents belonging to the authenticated user, optionally filtered by folder and/or filename search.	High
FR-DOC-06	The system SHALL allow authenticated users to download their own documents by UUID.	High
FR-DOC-07	The system SHALL allow authenticated users to delete their own documents; the filesystem file SHALL also be removed.	High
FR-DOC-08	The system SHALL prevent users from accessing documents belonging to other users.	High
FR-DOC-09	Users on the Free plan SHALL be limited to 3 PDF uploads per calendar day (client-enforced in	Medium

	v1.0).	

3.3 Folder Module

ID	Requirement	Priority
FR-FOL-01	The system SHALL allow authenticated users to create named folders (1–120 characters).	High
FR-FOL-02	The system SHALL prevent duplicate folder names per user (HTTP 409).	High
FR-FOL-03	The system SHALL list all folders belonging to the authenticated user in alphabetical order.	High
FR-FOL-04	The system SHALL allow deletion of a folder; documents inside it SHALL become uncategorised (folder_id = NULL).	High
FR-FOL-05	Users MUST select a folder before uploading a PDF.	High

3.4 Image Module

ID	Requirement	Priority
FR-IMG-01	The system SHALL allow authenticated users to create named image categories (1–120 characters).	High
FR-IMG-02	The system SHALL prevent duplicate category names per user (HTTP 409).	High
FR-IMG-03	The system SHALL allow image upload (JPEG, PNG, GIF, WEBP, BMP) up to 25 MB within a category.	High
FR-IMG-04	The system SHALL validate image MIME type and file extension on upload.	High
FR-IMG-05	The system SHALL serve image files with the correct MIME type via a dedicated endpoint.	High
FR-IMG-06	The system SHALL allow deletion of an image asset; the filesystem file SHALL also be removed.	High
FR-IMG-07	Deleting a category SHALL also delete all image assets within it.	High
FR-IMG-08	The system SHALL support filename search within a category.	Medium

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	All API endpoints (except /register and /login) SHALL require a valid JWT bearer token.
NFR-02	Security	Passwords SHALL be stored only as bcrypt hashes; plaintext passwords SHALL never be persisted.
NFR-03	Security	CORS SHALL be restricted to configured allowed origins (default: localhost:5173 and localhost:5174).
NFR-04	Performance	Document listing responses SHALL return within 500 ms for up to 1,000 documents per user.
NFR-05	Performance	File uploads SHALL be processed as streams to avoid loading entire files into memory.
NFR-06	Reliability	The backend SHALL roll back database transactions and delete partial files if an upload fails.
NFR-07	Usability	The frontend SHALL display loading indicators during all async operations.
NFR-08	Usability	All error responses SHALL be surfaced to the user with a human-readable message.
NFR-09	Maintainability	Configuration SHALL be fully externalized via environment variables (.env file).
NFR-10	Portability	The backend SHALL run on any OS with Python 3.10+; the frontend on any modern browser.
NFR-11	Scalability	The storage directory path SHALL be configurable to support external drives or network mounts.

5. System Architecture

5.1 Component Overview

The system follows a decoupled client-server architecture with the following components:

- React SPA (Frontend) — Served by Vite dev server on port 5173. Communicates with the backend via fetch() REST calls using a JWT Authorization header.
- FastAPI Application (Backend) — Runs on Uvicorn port 8000. Handles all business logic, file I/O, and database operations.

- SQLite Database — Single-file database stored at storage/pdf_vault.db. Managed via SQLAlchemy ORM.
- Filesystem Storage — PDF files stored in storage/pdfs/; image files in storage/pdfs/images/.

5.2 Database Schema

Table	Key Columns	Description
users	id (PK), username, email, password_hash, created_at	Registered user accounts
folders	id (PK), name, user_id (FK→users), created_at	User-created PDF folders
documents	id (PK), filename, stored_filename, size_bytes, created_at, user_id (FK), folder_id (FK)	PDF document records
image_categories	id (PK), name, user_id (FK→users), created_at	User-created image categories
image_assets	id (PK), filename, stored_filename, content_type, size_bytes, category_id (FK), user_id (FK), created_at	Image asset records

5.3 API Endpoints

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	No	Register a new user account
POST	/api/v1/auth/login	No	Login and receive JWT token
GET	/api/v1/auth/me	Yes	Get current user profile
GET	/api/v1/documents	Yes	List documents (supports search & folder_id filter)
POST	/api/v1/documents	Yes	Upload a PDF document
GET	/api/v1/documents/{id}/download	Yes	Download a PDF document
DELETE	/api/v1/documents/{id}	Yes	Delete a document
GET	/api/v1/documents/folders	Yes	List folders
POST	/api/v1/documents/folders	Yes	Create a folder
DELETE	/api/v1/documents/folders/{id}	Yes	Delete a folder
GET	/api/v1/images/categories	Yes	List image categories
POST	/api/v1/images/categories	Yes	Create image category
DELETE	/api/v1/images/categories/{id}	Yes	Delete image category (with images)
GET	/api/v1/images	Yes	List images in a category
POST	/api/v1/images	Yes	Upload an image to a category
GET	/api/v1/images/{id}/file	Yes	Serve image file
DELETE	/api/v1/images/{id}	Yes	Delete an image

6. External Interface Requirements

6.1 User Interface

- The frontend provides a Login/Register page, a Documents Dashboard, and an Images Workspace.
- Navigation between workspaces is achieved via a persistent sidebar.
- All interactive elements provide visual feedback (loading spinners, success notices, error banners).

6.2 Hardware Interfaces

No specific hardware interfaces are required beyond a standard computer with a web browser and network access.

6.3 Software Interfaces

- Frontend ↔ Backend: HTTP/1.1 REST API with JSON request/response bodies and multipart/form-data for file uploads.
- Backend ↔ Database: SQLAlchemy ORM using the sqlite:/// driver.
- Backend ↔ Filesystem: Python's pathlib.Path for cross-platform file operations.

7. Validation & Verification

Requirement ID	Test Type	Test Description
FR-AUTH-01	Integration	POST /register with valid data → HTTP 201 + token returned
FR-AUTH-03	Integration	POST /login with correct credentials → HTTP 200 + JWT returned
FR-AUTH-04	Integration	POST /login with wrong password → HTTP 401
FR-DOC-01	Integration	Upload 1 MB PDF → HTTP 201 + document record created
FR-DOC-02	Integration	Upload .txt file as PDF → HTTP 415
FR-DOC-02	Integration	Upload file with .pdf extension but no PDF header → HTTP 415
FR-DOC-07	Integration	DELETE /documents/{id} → HTTP 204 + file removed from disk
FR-FOL-02	Integration	Create two folders with same name → HTTP 409
FR-IMG-04	Integration	Upload .exe file as image → HTTP 415
NFR-03	Security	Attempt to access another user's document → HTTP 404